

M2: Initial Implementation — Unsupervised Detection of Attack Surface Drift

CS 4412 Data Mining

Alec Sundby

CS4412 ASM Data Mining, Kennesaw State University

asundby@student.kennesaw.edu

Abstract

This report summarizes the M2 implementation for my semester project on Attack Surface Management. I loaded and sampled approximately 78,500 authentication events from the LANL Unified Host and Network Dataset, parsed the raw logs into per-user behavioral profiles, and applied a PCA + DBSCAN pipeline to find natural groupings in the data. The clustering returned four distinct clusters with a silhouette score of 0.79 and a Davies-Bouldin index of 0.32. A rule-based drift proxy ($\text{failure_rate} > 0.15$ and $\text{ntlm_rate} > 0.50$) was used for post-hoc cluster validation rather than as an input to the algorithm. These results validate the discovery pipeline from M1 and set up the Isolation Forest anomaly scoring work for M3.

I. KEY EDA FINDINGS

A. Dataset and Event Distribution

The LANL 2017 auth dataset contains over 1.6 billion events. For this milestone I streamed and sampled approximately 78,500 records from the gzipped source file, covering 210 user identifiers over a 30-day window. Parsing the raw logs required mapping the LANL integer timestamp to an hour-of-day field and encoding the string auth-type and logon-type labels to numeric codes consistent with the downstream aggregation step.

The hourly distribution shows the expected business-hours peak between 7 and 18, but also a secondary concentration around 2–4 AM. That off-hours activity is disproportionately associated with elevated failure rates, which was one of the clearer early signals in the data.

B. Authentication Protocol Usage

Kerberos accounts for roughly 76% of events, which is expected for an Active Directory environment. NTLM sits at about 19%. That number matters because NTLM is the protocol most commonly involved in pass-the-hash and lateral movement scenarios. When I broke this down at the user aggregation step, `ntlm_rate` ended up being one of the stronger separating features.

C. Feature Correlation Structure

After aggregating to per-user profiles, the correlation matrix showed that `unique_src` and `unique_dst` are highly correlated — expected, since accounts that communicate broadly tend to both originate from and reach many hosts. The security-relevant features — `failure_rate`, `ntlm_rate`, and `off_hours_rate` — are relatively independent from the activity-volume features, so clustering results are not just driven by overall activity level.

II. PREPROCESSING DECISIONS

A. Feature Engineering

Raw event logs were aggregated into per-user behavioral profiles with 10 features: total events, unique source and destination machine counts, authentication failure rate, NTLM usage rate, interactive logon rate, off-hours activity rate, average and standard deviation of activity hour, and a lateral movement proxy (unique destinations per event). The `dst_per_event` feature was included because relative reach seemed more meaningful than raw destination count — an admin with 300 events and 150 unique destinations behaves differently from an account with the same counts but a different temporal pattern.

B. Missing Values and Scaling

No missing values were present after aggregation. The only edge case was `std_hour` for users appearing in a single event, which was filled with 0. Z-score standardization was applied before PCA and DBSCAN — without it, `total_events` would dominate every distance calculation and suppress the `failure_rate` and `ntlm_rate` signal.

III. MINING RESULTS

A. PCA

PCA on the 10 standardized features retained 4 components covering 97.7% of variance. The reduction is useful for two reasons: it addresses the curse of dimensionality that weakens DBSCAN's density estimates, and it allowed visual inspection of the data in PC1/PC2 space before committing to an epsilon value.

B. DBSCAN Parameter Selection

An epsilon sweep from 0.30 to 2.0 (`min_samples=4`) was run on the 4-component PCA output. At `eps=0.75`, the algorithm flagged 13 users as noise. Those accounts all had elevated failure rates and NTLM usage consistent with the drift proxy criteria, which is worth revisiting in M3 as a direct alternative to score-based detection. For the main run, `eps=1.2` was selected based on the highest silhouette score (0.7937) with 4 stable clusters.

C. Cluster Results

Table I summarizes the four clusters. The `drift_proxy` column cross-references the rule-based flag and was not used during clustering.

Cluster	N	Fail Rate	NTLM	Off-Hours	Avg Dst	Drift Proxy	Label
0	~150	~1.5%	~15%	~10%	~40	Low	Office Workers
1	~30	~0.6%	~10%	~95%	~85	Low	Service Accts
2	~20	~3.0%	~39%	~27%	~108	Low	IT Admins
3	varies	High	High	High	High	High	Drift / Anomalous

TABLE I. DBSCAN cluster summary (`eps=1.2`, `min_samples=4`, Silhouette=0.79, DBI=0.32). Drift proxy is rule-based post-hoc; not an input to clustering.

IV. INTERPRETATION AND M3 PLAN

A. What the Clusters Show

The four clusters map to recognizable operational archetypes. Cluster 0 is standard office activity — low failure rate, mostly Kerberos, concentrated during business hours. Cluster 1 is automated service accounts, identifiable by a near-zero failure rate and very high off-hours rate, consistent with scheduled tasks on a fixed cadence. Cluster 2 looks like IT admin or privileged accounts based on elevated NTLM usage and broader destination reach. Cluster 3 is where the drift proxy accounts land — substantially higher failure rate and NTLM usage than the rest. The clustering found this separation without any labels, just behavioral features from the raw auth logs.

B. Limitations

The lateral movement proxy (`dst_per_event`) was less discriminating than expected. Drift accounts and admin accounts overlap on that feature in the real data, which suggests that raw destination count — even normalized — may not be the right signal. Graph-based features like destination entropy or the ratio of new-to-seen machines per time window might do better. The current analysis also treats the 30-day window as static, so behavioral change over time within an individual account is not captured — which is what concept drift detection actually needs to address.

C. Updated M3 Plan

The M1 pipeline of PCA + DBSCAN + Isolation Forest is on track. M3 will apply the Isolation Forest to assign anomaly scores, particularly to accounts near or within the high-failure-rate cluster. I also plan to re-run DBSCAN at $\text{eps}=0.75$ to compare the noise-based outlier approach directly against the Isolation Forest scores. Additionally, I want to add time-windowed features by splitting the 30-day window into sub-periods to start capturing whether behavioral patterns shift within individual accounts over time.

REFERENCES

- [1] M. Turcotte, A. Kent, and C. Hash, “Unified Host and Network Data Set,” in *Data Science for Cyber-Security*, World Scientific, Nov. 2018, pp. 1–22. [Online]. Available: <https://csr.lanl.gov/data/2017/>
- [2] M. J. Zaki and W. Meira Jr., *Data Mining and Machine Learning*. Cambridge University Press, 2020. [Online]. Available: <https://dataminingbook.info>